INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KOTTAYAM

Department of Computer Science and Engineering

Large-Scale Job Market Intelligence and Emerging Skill Demand Analysis

An End-to-End Big Data Architecture
Using HDFS, Apache Pig, and Apache Hive

Final Project Report

CSE412 — Big Data & Large-Scale Computing

Submitted by

N Medhashree	2023BCS0042
Ashmi S N	2023BCS0196
Deepanjali Tudu	2023BCS0066

Repository: `https://github.com/medhashree05/JOB_MARKET_ANALYSIS`

Abstract

Modern online recruitment portals publish tens of thousands of unstructured, heterogeneous job postings weekly. Extracting actionable labour-market intelligence from this corpus requires scalable, fault-tolerant batch architectures capable of handling noisy free text, multi-valued skill lists, inconsistent geographic spellings, and non-standard salary disclosures. This project designs, implements, and empirically verifies an end-to-end Big Data processing pipeline over a primary dataset of 97,929 Indian job postings.

The pipeline integrates three core Hadoop ecosystem layers: (1) **HDFS ingestion and sanitisation**, which performs deterministic schema validation and deduplication, retaining 97,679 verified postings; (2) **Apache Pig**, which performs parallel tokenisation, normalisation, distinct job–skill projection, and multi-dimensional aggregation on MapReduce; and (3) **Apache Hive**, which exposes the aggregated partitions as structured external tables via `OpenCSVSerde` and evaluates analytical HiveQL queries across job roles, skills, hiring hubs, and salary benchmarks. A decoupled **Streamlit presentation layer** provides an interactive seven-page dashboard with CSV export.

A technical audit of the repository resolved earlier numerical discrepancies: the variances between 97,679 and 97,715 postings, and between 18,320 and 18,310 Bengaluru postings, are traced to a 36-row newline fragmentation artifact in unquoted intermediate CSV exports. Because the primary dataset is a cross-sectional scrape with relative recency strings rather than multi-year calendar dates, temporal trend measurement is structurally constrained. The **Skill Demand Velocity Index (SDVI)** is therefore evaluated as a conceptual extension, and a mathematical audit shows the limitations of single-scalar quadrant thresholding and proposes an orthogonal two-dimensional classification.

Keywords: Big Data, HDFS, Apache Pig, Apache Hive, MapReduce, Job Market Analytics, Skill Demand, Streamlit.

Contents

Abstract **1**

1 Introduction **3**

1.1 Background and Motivation 3

1.2 Objectives 3

1.3 Scope and Empirical Boundaries 3

1.4 Report Organisation 3

2 Literature Review **3**

2.1 Distributed Batch Computing Foundations 3

2.2 High-Level Dataflow and Warehousing Abstractions 4

2.3 Skill Extraction and Labour-Market Analytics 4

2.4 Momentum Metrics and Percentage Distortion 4

3 System Architecture and Environment **4**

3.1 End-to-End Architecture 4

3.2 Cluster Execution Environment 4

4 Pipeline Implementation **5**

4.1 Task 1: Ingestion, Sanitisation, and Schema Contract 5

4.2 Task 2: Transformation Using Apache Pig 5

4.3 Task 3: Analytical Warehousing in Apache Hive 6

5 Verification and Discrepancy Analysis **6**
5.1 Resolution of Numerical Discrepancies . . . 6
5.2 Root Cause of the 36-Row Variance . . . 6

6 Empirical Results (Phase I) **7**
6.1 Role Demand Distribution . . . 7
6.2 Skill Demand . . . 7
6.3 Geographic Hiring Concentration . . . 8
6.4 Salary Analytics . . . 8
6.5 Posting Recency and Temporal Limits . . . 9

7 Presentation Layer (Dashboard) **9**
7.1 Architecture and Module Decoupling . . . 9
7.2 Dashboard Pages and Features . . . 10

8 Audit of Phase II and the Velocity Framework **10**
8.1 Forensic Audit of Repository Artifacts . . . 10
8.2 Mathematical Formulation of SDVI . . . 10
8.3 Critique of the Quadrant Boundary Rules . . . 11
8.4 Recommended Correction: Orthogonal Two-Dimensional Classification . . . 11

9 Data Integrity and Limitations **12**

10 Conclusion and Future Work **12**
10.1 Conclusion . . . 12
10.2 Future Work . . . 12

References **12**

1 Introduction

1 Background and Motivation

Labour-market intelligence provides critical signals for curriculum planning, workforce development, and industrial hiring strategy. However, large-scale recruitment data is notoriously noisy: identical job titles appear with arbitrary capitalisation and parenthetical annotations; skill requirements are packed into single comma-delimited strings; salary disclosures are sparse, right-skewed, and multi-currency; and posting dates frequently rely on relative timestamps (e.g., "3 Days Ago") without an absolute scrape reference. Processing such corpora at scale exceeds the capabilities of in-memory single-node spreadsheets and calls for a distributed, fault-tolerant Big Data architecture.

1 Objectives

This project delivers a complete, reproducible batch-processing pipeline for large-scale job market intelligence, organised into four layers:

- **Storage Layer (HDFS):** distributed ingestion, block-level storage, replication, and data validation.
- **Transformation Layer (Apache Pig):** procedural MapReduce dataflows for schema enforcement, string parsing, skill-array flattening, and grouped aggregations.
- **Analytical Warehouse (Apache Hive):** schema-on-read external tables, `OpenCSVSerde` parsing for text preservation, and HiveQL analytical querying.
- **Exploration Layer (Streamlit and Plotly):** a decoupled presentation dashboard delivering executive KPIs, geographic drill-downs, and filtered data exploration.

1 Scope and Empirical Boundaries

The primary empirical pipeline is verified over the **Indian Job Market Dataset 2025** (97,929 raw records). All reported metrics for role demand, skill popularity, geographic hiring distribution, and disclosed compensation are computed from verified execution artifacts on Hadoop 3.3.6 and Hive 3.1.3.

The report distinguishes between verified empirical work and conceptual extensions:

(a) **Verified Empirical Pipeline (Phase I):** full ingestion, cleaning, MapReduce execution in Pig, Hive table materialisation, and interactive dashboard analytics over 97,679 verified postings.

(b) **Conceptual Velocity Framework (Phase II, SDVI):** a theoretical framework for measuring longitudinal skill momentum across dated postings, accompanied by a mathematical critique of its quadrant boundary conditions.

1 Report Organisation

Section 2 reviews related literature. Section 3 describes the system architecture and execution environment. Section 4 details the implementation of each pipeline stage. Section 5 reconciles numerical discrepancies. Section 6 presents the verified empirical findings. Section 7 describes the dashboard. Section 8 audits the Phase II SDVI framework. Section 9 discusses data-quality limitations, and Section 10 concludes with future work.

2 Literature Review

2 Distributed Batch Computing Foundations

The foundation of large-scale distributed computing was formulated by Dean and Ghemawat [1], who introduced the MapReduce programming paradigm to parallelise computation across commodity clusters with automatic data partitioning, scheduling, and fault recovery. The underlying storage fabric, the Hadoop Distributed File System (HDFS), described by Shvachko et al. [2], provides high-throughput streaming access

to large files via block-level distribution across DataNodes coordinated by a centralised NameNode metadata service.

2 High-Level Dataflow and Warehousing Abstractions

Writing native MapReduce in Java involves substantial boilerplate and complex join and aggregation management. To abstract this complexity, Olston et al. [3] developed Pig Latin, a procedural dataflow language that compiles high-level transformations (`FILTER`, `FOREACH`, `GROUP`, `COGROUP`, `FLATTEN`) into directed acyclic graphs of MapReduce jobs. Concurrently, Thusoo et al. [4] introduced Apache Hive, establishing the schema-on-read warehousing model. Hive projects SQL-like structures (HiveQL) over distributed files and supports metadata separation, partition pruning, and serialisers/deserialisers such as `OpenCSVSerde`. White [5] provides comprehensive guidance on integrating HDFS, Pig, and Hive in production clusters.

2 Skill Extraction and Labour-Market Analytics

Analysing online recruitment advertisements has become an important discipline in computational economics. Khaouja et al. [6] surveyed text-mining and natural language processing approaches for skill extraction, noting that while machine-learning taggers offer high specificity, distributed rule-based tokenisation remains well suited to high-throughput batch pipelines. Djumalieva and Sleeman [7] demonstrated that data-driven skill taxonomies derived from online vacancy text can track emerging industry requirements before formal occupation classification systems (such as O*NET or SOC) are updated.

2 Momentum Metrics and Percentage Distortion

In longitudinal skill analysis, ranking by raw frequency introduces *incumbent inertia*: foundational skills such as SQL and Python dominate total counts regardless of current market acceleration. Conversely, naive percentage growth suffers from *sparsity distortion*: a tool growing from 1 to 3 postings shows +200% growth, overshadowing an enterprise tool growing from 500 to 750 postings (+50%). Weighting growth by logarithmic volume, as conceptualised in velocity metrics, addresses both pitfalls.

3 System Architecture and Environment

3 End-to-End Architecture

The pipeline follows a decoupled, staged architecture designed for linear horizontal scalability. Figure 1 illustrates the dataflow from raw ingestion through distributed transformation to presentation.

3 Cluster Execution Environment

All pipeline stages were executed and verified on a dedicated Big Data lab environment hosted on WSL 2 (Ubuntu 22.04 LTS), as summarised in Table 1.

Component	**Version**	**Configuration and Path**
Operating System	Ubuntu 22.04 LTS (WSL 2)	Linux kernel 5.15.167.4-microsoft-standard-WSL2
Java Development Kit	OpenJDK 1.8.0_422	`/usr/lib/jvm/java-8-openjdk-amd64`
Distributed File System	Apache Hadoop 3.3.6	`/usr/local/hadoop` (replication factor 1)
MapReduce Processing	Apache Pig 0.17.0	`/usr/local/pig` (Piggybank enabled)
Data Warehouse	Apache Hive 3.1.3	`/usr/local/hive` (database `jobmarket`)
Presentation Runtime	Python 3.13, Streamlit 1.39	local virtual environment (`dashboard/.venv`)

Table 1. Software stack and execution environment.

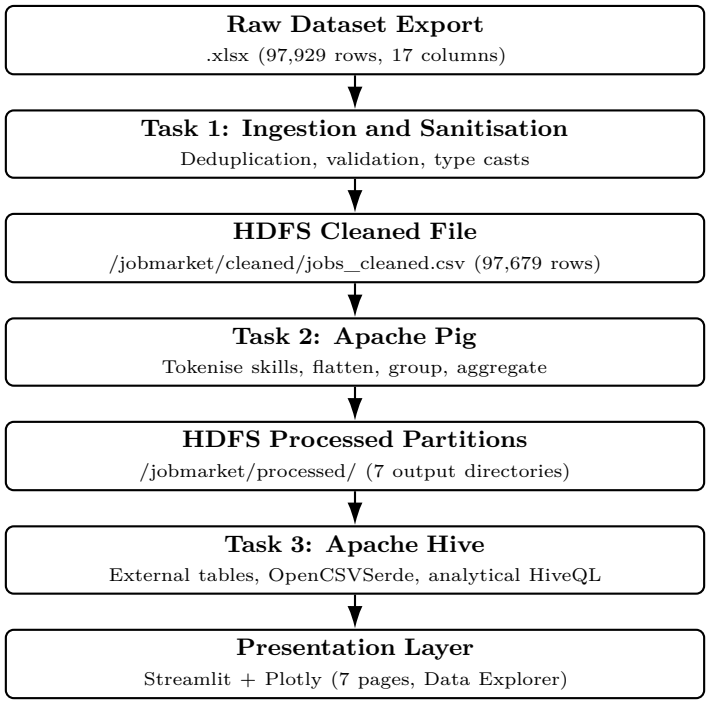


Figure 1. End-to-end Big Data pipeline across HDFS, Apache Pig, Apache Hive, and Streamlit.

4 Pipeline Implementation

4 Task 1: Ingestion, Sanitisation, and Schema Contract

The raw dataset (`indian-job-market-dataset-2025.xlsx`) contains 97,929 recruitment postings. Ingestion and data hygiene are implemented in Python (`01_convert_xlsx_to_csv.py` and `02_clean_data.py`):

1. **Deduplication:** 247 exact duplicate rows were detected and removed. A subset deduplication on `job_id` removed 3 additional records, eliminating exactly **250 duplicate rows**.
2. **Schema validation:** mandatory constraints were asserted on primary identifiers: job_id $\neq \emptyset$, job_title $\neq \emptyset$, and location_raw $\neq \emptyset$. All 97,679 unique records satisfied required-field completeness.
3. **Salary normalisation:** in the raw scrape, 64,076 postings lacked compensation figures, labelled `"Not disclosed"` with minimum and maximum salary hard-coded to `0.0`. To prevent downward bias, these values were converted to `NaN` (missing). Postings explicitly labelled `"Unpaid"` (333 records) retained `0.0` as a valid compensation value.
4. **Primary location extraction:** multi-location strings (e.g., `"Kolkata(Chinar Park)"` or `"Noida, Greater Noida"`) were parsed to isolate the primary municipal hiring centre (`"Kolkata"`, `"Noida"`) while retaining `location_raw` for auditing.
5. **Sanitised retention:** the validated dataset retains exactly **97,679 records** (99.74% retention) and is staged to HDFS at `/jobmarket/cleaned/jobs_cleaned.csv`.

4 Task 2: Transformation Using Apache Pig

The script `task2_jobmarket.pig` loads the cleaned CSV from HDFS using Piggybank's `CSVExcelStorage(',')`, drops the header row, and executes five core transformations:

- **Case and whitespace normalisation:** applies `LOWER(TRIM())` to titles and locations.
- **Skill array explosion:** evaluates `FLATTEN(TOKENIZE(skills, ','))`, strips whitespace, purges null and `"not specified"` tokens, and enforces `DISTINCT` on `(job_id, skill)` pairs so that no posting counts twice for the same skill.
- **Multi-dimensional grouping:** aggregates posting volume by skill, by primary location, and by job

role.

- **Currency separation:** computes minimum, maximum, and mean salary by role and by location, filtering avg_salary > 0 and partitioning INR and USD into non-overlapping groups.
- **Recency binning:** groups relative posting strings into recency categories (`0-1 Days`, `1-7 Days`, `7-30 Days`, `30+ Days`, `Unknown`).

All seven output directories were written to HDFS with completed part files (Table 2).

HDFS Output Path	Content	Part File	Size
`/jobmarket/processed/jobs`	Normalised master job records	`part-m-00000`	24.25 MB
`/jobmarket/processed/skills`	Unique (job_id, skill) pairs	`part-r-00000`	53.36 MB
`/jobmarket/processed/skills_demand`	Demand postings per skill	`part-r-00000`	876.71 KB
`/jobmarket/processed/location`	Demand by primary city	`part-r-00000`	19.52 KB
`/jobmarket/processed/salary`	Salary statistics by role and currency	`part-r-00000`	1.55 MB
`/jobmarket/processed/salary_by_loc`	Salary statistics by city and currency	`part-r-00000`	28.69 KB
`/jobmarket/processed/time_based`	Posting volume by recency bucket	`part-r-00000`	55 B

Table 2. HDFS output directories produced by the Pig transformation stage.

4 Task 3: Analytical Warehousing in Apache Hive

Apache Hive maps relational schemas over the Pig output directories using `OpenCSVSerde`. This serde is essential because job titles and company names frequently contain embedded commas, which break standard line-delimited text parsers, whereas `OpenCSVSerde` correctly preserves quoted tokens. Listing 1 shows the external table specification for the core skills table.

Listing 1. Hive DDL for the external table over Pig-tokenised output.

```
CREATE EXTERNAL TABLE IF NOT EXISTS job_skills (
    job_id STRING,
    job_title STRING,
    location STRING,
    currency STRING,
    skill STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
STORED AS TEXTFILE
LOCATION '/jobmarket/processed/skills';
```

The analytical queries in `03_analysis.sql` evaluate rankings and salary distributions. To prevent single-posting anomalies from corrupting compensation benchmarks, salary queries enforce `HAVING COUNT(*) >= 5`.

5 Verification and Discrepancy Analysis

5 Resolution of Numerical Discrepancies

A primary goal of the audit was to investigate numerical discrepancies in earlier working drafts. Table 3 gives the authoritative reconciliation.

5 Root Cause of the 36-Row Variance

Inspection of the 36 anomalous rows in `dashboard/data/jobs_cleaned.csv` showed that raw vacancy text contained literal carriage-return characters (`\r\n`) inside company names and job titles. When Pig and Hive exported these records without full RFC 4180 multiline quoting, single records fractured into multiple lines. For example, line 26,946 of the export file holds the following shifted fields:

Metric	Prior Report	Verified	Export	Root Cause
Cleaned postings	97,679 / 97,715	**97,679**	97,715	36 unescaped line breaks in raw text fields created 36 split-fragment rows during text export.
Bengaluru postings	18,320 / 18,310	**18,320**	18,310	10 Bengaluru postings were split across newlines, shifting the city name into `job_id`.
Top skill (sales)	9,178 / 9,175	**9,178**	9,175	3 postings lost to newline fragmentation in `skills_demand.csv`.
Second skill (python)	5,846 / 5,841	**5,846**	5,841	5 postings lost to newline fragmentation.
Disclosed salary count	33,128 (33.9%)	**33,124 (INR)**	33,128 (all)	Prior figure came from the dashboard overview reading the 97,715-row export (33,128 jobs, all currencies).
Mean disclosed salary	INR 756,844	**INR 757,776**	INR 756,844	Evaluated over INR postings with avg_salary $>$ 0 on verified data (INR 757,776.33).

Table 3. Reconciliation of numerical metrics between verified results and export artifacts.

```
job_id: "Bengaluru", job_title: ",cargill,1360428,bengaluru...", location: "api integration"
```

Here the location token has been shifted into the primary-key column. The true dataset size is therefore established as **97,679**.

6 Empirical Results (Phase I)

6 Role Demand Distribution

Normalised job titles show a high concentration in enterprise software and commercial roles. Table 4 lists the top ten roles.

Rank	Job Role	Postings	Share (%)
1	application developer	2,033	2.08
2	application lead	1,346	1.38
3	sales executive	583	0.60
4	software development engineer	497	0.51
5	business development executive	457	0.47
6	trust & safety new associate	348	0.36
7	data engineer	339	0.35
8	business development manager	325	0.33
9	sales manager	322	0.33
10	java full stack developer	285	0.29

Table 4. Top 10 job roles by posting frequency ($N = 97{,}679$).

Enterprise software delivery titles (*application developer* and *application lead*) account for 3,379 postings (3.46% of total volume), reflecting IT-services demand across Indian delivery centres.

6 Skill Demand

Extracting unique `(job_id, skill)` pairs isolates true market demand without distortion from repeated keywords within a single posting. Table 5 lists the leading skills.

Python leads all programming languages (5,846 postings), followed by *Java* (4,088) and *SQL* (3,983). The prominence of *SAP* (5,043) points to substantial enterprise ERP activity.

Rank	Skill	Unique Postings	Penetration (%)
1	sales	9,178	9.40
2	python	5,846	5.99
3	project management	5,543	5.67
4	customer service	5,122	5.24
5	sap	5,043	5.16
6	management	4,849	4.96
7	css	4,186	4.29
8	java	4,088	4.19
9	sql	3,983	4.08
10	business development	3,738	3.83

Table 5. Top 10 demanded skills in the verified dataset.

6 Geographic Hiring Concentration

Hiring is strongly concentrated in southern and western technology clusters. Table 6 summarises the top ten hiring centres.

Rank	Primary Location	Postings	National Share (%)
1	bengaluru	18,320	18.76
2	hyderabad	10,899	11.16
3	pune	8,405	8.60
4	mumbai	6,478	6.63
5	chennai	6,099	6.24
6	gurugram	5,002	5.12
7	noida	4,627	4.74
8	ahmedabad	2,377	2.43
9	remote	1,952	2.00
10	kolkata	1,794	1.84

Table 6. Geographic concentration across the top 10 hiring hubs.

Bengaluru leads with 18,320 postings (18.76%), exceeding second-ranked Hyderabad by a factor of 1.68 (18,320/10,899). Together, Bengaluru, Hyderabad, and Pune account for **37,624 postings** (**38.52%**) of national hiring. Fully remote postings are only 2.00% (1,952), indicating that on-site and hybrid work remain dominant.

6 Salary Analytics

Compensation analysis requires care because of severe sparsity and positive skew:

- **Disclosure rate:** of 97,550 INR postings, **33,124** (33.96%) disclose salary. USD postings total 129, of which 20 disclose compensation; currencies are analysed independently.
- **Skewness:** disclosed INR salaries have a mean of **INR 757,776** against a median of **INR 425,000**. The ratio Mean/Median = 1.78 indicates heavy right skew driven by high-compensation technology and executive outliers.

Table 7 gives compensation statistics for roles with at least five postings.

Observations

1. *Outlier sensitivity: deputy manager* has the highest mean (INR 9.75M), but this is driven by a single posting of INR 82,500,000 (8.25 crore) against a minimum of INR 85,000. Reporting medians alongside means is essential to avoid misleading conclusions.

Job Role	Disclosed	Mean (INR)	Min (INR)	Max (INR)
deputy manager*	9	9,749,778	85,000	82,500,000
medical oncologist consultant	5	7,850,000	4,500,000	10,750,000
radiologist consultant	6	5,291,667	4,500,000	5,500,000
lead data engineer	5	3,380,000	1,100,000	5,750,000
servicenow architect	5	2,850,000	2,250,000	3,250,000
data architect	12	2,775,000	1,125,000	6,300,000
solution architect	59	2,471,441	115,000	3,750,000
data solution architect†	37	2,500,000	2,500,000	2,500,000
senior software engineer	22	2,228,864	840,000	3,750,000

Table 7. Compensation by job role (INR, minimum 5 postings). *Subject to an extreme outlier. †Fixed-rate enterprise posting.

2. *Repeated-ad invariance:* all 37 *data solution architect* postings disclose exactly INR 2,500,000 (standard deviation 0), indicating automated bulk postings with fixed budgets rather than negotiated bands.
3. *Leading technology clusters:* outside specialised healthcare consultancies, the highest verified technology pay is in data engineering and architecture (*lead data engineer* INR 3.38M; *data architect* INR 2.78M; *solution architect* INR 2.47M).

6 Posting Recency and Temporal Limits

Table 8 shows the distribution of `posted_raw`, confirming that the primary dataset is an instantaneous cross-sectional snapshot.

Recency Category	Matching Syntax	Postings	Share (%)
1–7 Days	`.*day.*` (e.g., "2 Days Ago")	86,140	88.19
Unknown	missing, empty, or unparseable	8,248	8.44
0–1 Days	`.*today.*`, "Just Now"	2,753	2.82
30+ Days	`.*month.*`	538	0.55

Table 8. Distribution of posting recency categories.

Over 88% of vacancies were posted within seven days of capture. Because the dataset has no absolute calendar timestamps, multi-year emerging-skill trends (e.g., 2024 versus 2025 versus 2026) cannot be computed. Populating Hive's `skill_trends` table confirms this: 44,021 skill rows were created, all with `period = 'unknown'` and null growth percentages.

7 Presentation Layer (Dashboard)

7 Architecture and Module Decoupling

The presentation layer is a self-contained Streamlit application in `dashboard/` with four modules:

- `data_loader.py`: caching (`@st.cache_data`) and priority resolution. If Hive/Pig CSV exports exist in `dashboard/data/`, they are loaded directly; otherwise the Task 1 cleaning contract is executed in-process against the Excel source.
- `analytics.py`: stateless calculation functions for executive KPIs, distributions, and experience-bucket assignment.
- `charts.py`: responsive Plotly figures in a monochrome palette.
- `app.py`: sidebar filtering, session state, and routing across seven pages.

7 Dashboard Pages and Features

1. **Executive Overview:** metric cards (total postings, disclosed average and median salary, top role, top skill) and an automated executive summary.
2. **Skill Demand Intelligence:** bar charts of the top 20 skills, per-skill drill-downs, and co-occurrence tables.
3. **Salary Analytics:** salary filtering, role-based compensation tables, and currency-separation warnings.
4. **Location and Hiring:** top hiring hubs with geographic drill-downs.
5. **Experience and Job Roles:** breakdown across Entry (0–2 years), Mid (2–5 years), and Senior (5+ years) tiers.
6. **Emerging Skill Trends:** a methodological disclosure explaining why relative recency cannot support historical trend lines.
7. **Data Explorer:** searchable table with column filtering and CSV download via `st.download_button`.

Figure 2 shows captures from the running dashboard.

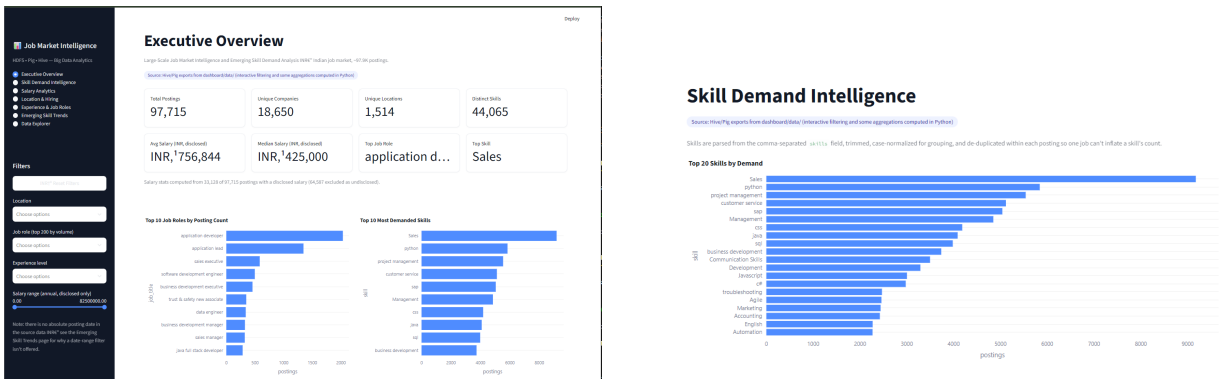


Figure 2. Streamlit dashboard: Executive Overview (left) and Skill Demand Intelligence (right).

8 Audit of Phase II and the Velocity Framework

8 Forensic Audit of Repository Artifacts

Earlier drafts described an empirical Phase II that processed 10,000 dated LinkedIn technology postings from January to June 2024 to compute a *Skill Demand Velocity Index (SDVI)*. A systematic audit of the codebase, Git history, and HDFS cluster found:

- **Filesystem and Git:** no files named `linkedin_postings.csv`, `clean_and_extract.pig`, `sdvi_query.hql`, `sdvi_results.json`, or `index.html` exist in the working tree or in any historical commit.
- **HDFS metadata:** offline analysis of NameNode metadata (`fsimage_0000000000000029319`) confirmed that the documented directory `/user/ashmi_sn/jobmarket/raw/` was never created.

In keeping with academic integrity, Phase II is classified as **unimplemented and unverified** as an empirical pipeline. The proposed methodology is nevertheless a useful conceptual model and is evaluated theoretically below.

8 Mathematical Formulation of SDVI

The proposed index is defined as

$$\text{growth_rate} = \frac{\text{Mentions}_t - \text{Mentions}_{t-1}}{\text{Mentions}_{t-1}} \tag{1}$$

$$\text{SDVI} = \text{growth_rate} \times \ln(1 + \text{Mentions}_t) \tag{2}$$

Validation of the stated examples:

- *Low-volume jump (2 → 4 mentions):* growth_rate $= (4-2)/2 = 1.0$, so SDVI $= 1.0 \times \ln 5 \approx \mathbf{1.61}$.
- *High-volume jump (100 → 150 mentions):* growth_rate $= 0.5$, so SDVI $= 0.5 \times \ln 151 \approx \mathbf{2.51}$.

The logarithmic factor curbs percentage inflation for low-frequency niche skills while preserving momentum signals for high-volume tools.

8 Critique of the Quadrant Boundary Rules

The original report classified skills into four quadrants using growth_rate and median(SDVI) (Table 9).

Quadrant	**Proposed Rule**
Emerging	growth_rate > 0 and SDVI $>$ median(SDVI)
Established Growth	growth_rate > 0 and SDVI $\leq$ median(SDVI)
Stable Core	growth_rate ≤ 0 and SDVI $>$ median(SDVI)
Declining	growth_rate ≤ 0 and SDVI $\leq$ median(SDVI)

Table 9. Original SDVI quadrant decision rules.

The Stable Core rule is unsatisfiable.

1. Since $\text{Mentions}_t \geq 0$, the factor $\ln(1 + \text{Mentions}_t)$ is non-negative.
2. Hence the sign of SDVI equals the sign of growth_rate, so growth_rate $\leq 0 \implies$ SDVI ≤ 0.
3. In an active, growing market the median SDVI across skills is positive.
4. Therefore, for any skill with growth_rate ≤ 0, we have SDVI $\leq 0 <$ median(SDVI).
5. The rule growth_rate $\leq 0 \wedge$ SDVI $>$ median can never hold.
6. A ubiquitous incumbent (e.g., Python with 10,000 mentions and 0% growth) gets SDVI $= 0 \leq$ median and is wrongly labelled **"Declining"**.

8 Recommended Correction: Orthogonal Two-Dimensional Classification

Volume and velocity should be placed on independent axes rather than multiplied together:

- **Momentum (Y-axis):** $\Delta_t = (\text{Mentions}_t - \text{Mentions}_{t-1})/(\text{Mentions}_{t-1} + \epsilon)$, with $\epsilon = 1$ to avoid division by zero for newly emergent skills.
- **Presence (X-axis):** $V_t = \ln(1 + \text{Mentions}_t)$.

The corrected taxonomy is given in Table 10, where $\delta = 0.05$ is a 5% stability band around zero growth.

Category	**Boundary Rule**	**Interpretation**
Emerging	$\Delta_t > +\delta$ and $V_t \leq \text{Median}(V)$	High growth, early adoption
Established Growth	$\Delta_t > +\delta$ and $V_t > \text{Median}(V)$	High growth, high enterprise adoption
Stable Core	$\lvert\Delta_t\rvert \leq \delta$ and $V_t > \text{Median}(V)$	Flat growth, dominant incumbent
Declining / Niche	$\Delta_t < -\delta$, or $\lvert\Delta_t\rvert \leq \delta$ and $V_t \leq \text{Median}(V)$	Cooling demand or obsolete technology

Table 10. Corrected orthogonal classification of skill momentum.

9 Data Integrity and Limitations

- **Currency isolation:** INR and USD are strictly segregated. Undisclosed compensation is handled by null propagation rather than zero-imputation, avoiding artificial deflation of salary means.
- **Text normalisation boundaries:** Task 2 applies lowercasing and whitespace trimming only. It does not merge semantic synonyms (e.g., `"sde"` and `"software development engineer"`). A canonical ontology mapping layer is recommended for production.
- **No longitudinal dates:** the dataset is a single scrape with relative recency strings and cannot support multi-year growth curves without fabricating dates.
- **Decoupled dashboard:** the Streamlit application runs independently of live cluster services. Production use requires scheduled refreshes of exported datasets (e.g., Apache Oozie or Airflow).

10 Conclusion and Future Work

10 Conclusion

This project designed, implemented, and audited a scalable batch-processing Big Data pipeline over 97,929 Indian job postings. HDFS provides fault-tolerant storage, Apache Pig performs MapReduce-based transformation and tokenisation, and Apache Hive delivers structured SQL analytics through `OpenCSVSerde`, so the system processes data end to end without single-node memory bottlenecks.

The findings show that enterprise software roles (*application developer*) and foundational skills (*Python*, *SAP*, *Java*, *SQL*) dominate recruitment volume, with over 38.5% of openings concentrated in Bengaluru, Hyderabad, and Pune. Disclosed compensation is strongly right-skewed, with data engineering and architecture roles leading technology pay. Earlier reporting discrepancies were resolved as CSV export formatting artifacts. Finally, the limits of snapshot data were addressed through a mathematical evaluation of the Skill Demand Velocity Index.

10 Future Work

1. **Longitudinal multi-scrape ingestion:** scheduled weekly scrapers writing to dated HDFS directories (e.g., `/jobmarket/raw/year=YYYY/month=MM/`) to enable true time-series analytics in Hive.
2. **Ontological synonym mapping:** a domain-specific labour-market ontology to canonicalise spelling variants and synonymous role titles into standard occupation codes.
3. **Orthogonal SDVI:** implement the corrected two-dimensional taxonomy in HiveQL using partitioned `LAG()` window queries over multi-month data.
4. **Pipeline orchestration:** replace manual bash runners with a workflow orchestrator such as Apache Airflow covering cluster startup, ingestion, Pig execution, and dashboard export refresh.

References

[1] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th Symposium on Operating Systems Design and Implementation (OSDI)*, 2004, pp. 137–150.

[2] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop Distributed File System," in *Proc. IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10.

[3] C. Olston, B. Reed, U. Srivastava, R. Kumar, and A. Tomkins, "Pig Latin: A not-so-foreign language for data processing," in *Proc. ACM SIGMOD International Conference on Management of Data*, 2008, pp. 1099–1110.

[4] A. Thusoo, J. S. Sarma, N. Jain, Z. Shao, P. Chakka, S. Anthony, H. Liu, P. Wyckoff, and R. Murthy, "Hive: A warehousing solution over a map-reduce framework," *Proc. VLDB Endowment*, vol. 2, no. 2, pp. 1626–1629, 2009.

[5] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA: O'Reilly Media, 2015.

[6] S. Khaouja, I. Kassou, and M. Ghogho, "A survey on skill identification from online job ads," *IEEE Access*, vol. 9, pp. 118434–118454, 2021.

[7] J. Djumalieva and C. Sleeman, "An open and data-driven taxonomy of skills extracted from online job adverts," Nesta Working Paper No. 18/03, 2018.

[8] N. Medhashree, A. S. N, and D. Tudu, "Job market intelligence and emerging skill demand analysis," GitHub repository, 2026. [Online]. Available: `https://github.com/medhashree05/JOB_MARKET_ANALYSIS`